Go-reloaded

Documentation du projet · README corrigé et mis en page

Présentation

Go-reloaded est un programme en ligne de commande qui lit un fichier texte, applique des règles de conversion et de formatage, puis écrit le résultat dans un fichier de sortie. Le fichier d'entrée reste inchangé.

Le texte est traité ligne par ligne. Les espaces multiples d'une ligne sont normalisés en espaces simples dans le résultat.

Origine du projet

Ce dépôt met en œuvre le sujet Go-reloaded. Des fonctionnalités supplémentaires inspirées d'une partie du sujet Go Reforged sont séparées dans <link href="fonction/bonus/readme.md" color="#2563EB">fonction/bonus/readme.md</link>.

Prérequis

- Go 1.27.1, ou une version compatible avec celle déclarée dans go.mod.
- Git uniquement pour cloner le dépôt.

Le projet n'utilise que la bibliothèque standard de Go et ne nécessite aucune configuration externe.

Installation

```
git clone https://github.com/Mathiazfnt/Go-reloaded.git
cd Go-reloaded
```

Utilisation

Depuis la racine du dépôt, indique le fichier d'entrée puis le fichier de sortie :

```
go run . sample.txt result.txt
```

Le fichier de sortie est créé s'il n'existe pas et remplacé s'il existe déjà. Pour compiler puis lancer le programme :

```
go build -o go-reloaded.exe .
.\go-reloaded.exe sample.txt result.txt
```

Le programme attend exactement deux chemins en argument. Il affiche une instruction d'utilisation si le nombre d'arguments est incorrect.

Règles prises en charge

Les commandes listées ci-dessous décrivent les règles de Go-reloaded. Les fonctionnalités ajoutées dans le dossier fonction/bonus/ sont présentées dans son README dédié.

Règle	Exemple d'entrée	Résultat
Hexadécimal vers décimal	`2a (hex)`	`42`
Binaire vers décimal	`1010 (bin)`	`10`
Tout en majuscules	`bonjour (up)`	`BONJOUR`

Règle	Exemple d’entrée	Résultat
Tout en minuscules	`BONJOUR (low)`	`bonjour`
Première lettre en majuscule	`bonjour (cap)`	`Bonjour`
Ajuster `a` / `an`	`a apple and an banana`	`an apple and a banana`
Rapprocher la ponctuation	`bonjour ,monde !`	`bonjour, monde!`
Points et `!?`	`attends .. vraiment !?`	`attends.. vraiment!?`
Apostrophes et guillemets	`' bonjour '`	`'bonjour'`

Pour (up), (low) et (cap), un nombre facultatif indique combien de mots précédents traiter, par exemple (up, 2).

Les règles sont appliquées dans l’ordre défini dans main.go.

Limites connues

- L’ajustement de a et an examine seulement le premier caractère du mot suivant et utilise une liste limitée aux voyelles ASCII et à h.
- La règle des points de suspension colle actuellement deux points (`..`), même lorsque l’entrée en contient trois.
- Les nombres invalides pour les conversions hexadécimales ou binaires peuvent rester inchangés.
- Les fonctionnalités supplémentaires inspirées de Go Reforged sont documentées dans le README du dossier bonus.

Exemple fourni

sample.txt contient des exemples des règles principales et des fonctionnalités bonus. result.txt contient le résultat généré par cette commande :

```
go run . sample.txt result.txt
```

Tests

Exécuter les tests et vérifier la compilation de tous les packages avec :

```
go test ./...
```

Architecture

```
.
■■■ main.go
■■■ sample.txt
■■■ result.txt
■■■ fonction/
    ■■■ tableau.go
    ■■■ tableau_test.go
    ■■■ casse_mots/
    ■   ■■■ cap.go
    ■   ■■■ cap_test.go
    ■   ■■■ low.go
    ■   ■■■ low_test.go
    ■   ■■■ upp.go
    ■   ■■■ upp_test.go
    ■   ■■■ depassement.go
    ■   ■■■ depassement_test.go
    ■■■ conversions_numeriques__a_an/
    ■   ■■■ a_an.go
    ■   ■■■ a_an_test.go
    ■   ■■■ bin.go
    ■   ■■■ bin_test.go
    ■   ■■■ hex.go
    ■■■ ponctuation/
        ■■■ apostrophes.go
        ■■■ apostrophe_test.go
        ■■■ ponctuation_spl.go
        ■■■ ponctuation_spl_test.go
        ■■■ exclama_inter.go
        ■■■ exclama_inter_test.go
```

- `main.go` lit le fichier d'entrée, applique les règles et écrit le résultat.
- `fonction/tableau.go` regroupe les mots d'une ligne et conserve ensemble les commandes entre parenthèses.
- `fonction/casse_mots/` contient les règles de casse et le calcul du nombre de mots à traiter.
- `fonction/conversions_numeriques__a_an/` contient les conversions hexadécimale et binaire vers décimal et l'ajustement de a et an.
- `fonction/ponctuation/` regroupe le traitement de la ponctuation, des apostrophes et des guillemets.
- Les fonctionnalités bonus issues d'une partie de Go Reforged sont décrites dans `fonction/bonus/readme.md`.